

Soft-Settle

Micro-Credit Line

Vision-

It is a system designed to enable high-speed, low-trust micro-transactions between AI agents in B2B networks. It addresses the limitations of on-chain settlements for streaming services by combining off-chain state channels, decentralized reputation via ENS, and final on-chain settlements using Circle's infrastructure. The system facilitates "soft" off-chain authorizations during service delivery and "hard" on-chain settlements upon completion, reducing latency and gas costs while maintaining security.

Problem Statement-

On-chain blockchain settlements are too slow and expensive for granular, streaming services like real-time data scraping. AI agents require pre-work payment proofs without risking rug-pulls, enabling trust-minimized commerce.

Technical Overview-

The Soft-Settle Micro-Credit Line is a decentralized system for enabling micro-transactions between AI agents in B2B networks, leveraging off-chain state channels for low-latency "soft" settlements and on-chain mechanisms for final "hard" settlements. It integrates with Yellow Network (Nitrolite state channels), Circle/Arc (for USDC payouts and credit management), and ENS (for reputation storage).

The core innovation is a "trust tunnel" that allows agents to stream micro-authorizations off-chain via cryptographically signed messages, minimizing blockchain interactions to only session open/close events.

The system operates on Ethereum-compatible chains, using smart contracts for collateral locking, state enforcement, and reputation management. Off-chain components handle real-time communication via P2P protocols. Security relies on EIP-712 signatures for state updates, nonces to prevent replays, and a challenge period for unilateral closures to handle disputes.

The architecture ensures scalability for high-frequency interactions while keeping gas costs low. We will build a web application that serves as a hub for agent registration, management, and interaction. The site will act as a dashboard for developers/human overseers to onboard agents, monitor sessions, and configure parameters, while providing APIs for agents to interact autonomously. Agents themselves will primarily use backend APIs, but the site will include tools for simulating agent behaviors during development.

Technical pillars-

Blockchain Layer: Solidity smart contracts for Nitrolite channels, Reputation Manager, and integration hooks for Circle APIs.

Off-Chain Layer: Node.js or Go-based microservices for P2P messaging, signature validation, and session orchestration.

Frontend Layer: React.js with Web3.js/Ethers.js for wallet interactions, real-time data via GraphQL subscriptions.

Backend Layer: API server (e.g., Express.js) for agent registration, session queries, and Circle/ENS integrations.

Data Storage: IPFS for off-chain logs (optional for audits), The Graph for indexing on-chain events.

Security Features: Multi-signature for critical actions, rate limiting on APIs, and zero-knowledge proofs (future enhancement) for private settlements.

Scalability: Horizontal scaling of off-chain nodes; L2 rollups for on-chain ops to handle 1,000+ concurrent sessions.

User Workflow-

Agent Registration (On Website):

- Developer navigates to the Soft-Settle website and connects their Ethereum wallet.
- UI: Clean landing page with a "Register Agent" CTA, featuring a form for ENS domain, agent description, and API endpoint for callback notifications.
- Backend: Submits to API, which registers the agent in a database and initializes ENS text records.
- Agent Autonomy: Once registered, the agent can self-authenticate via signed messages; no human intervention needed for future sessions.
- UI Elements: Dashboard with agent list, editable profiles, and real-time status badges. Inspired by ENS app's domain management interface.

Credit Check and Session Initiation (Agent-to-Agent, Monitored via Website):

- Consumer agent queries provider's API to request service.
- Provider queries consumer's ENS records via API call to the website's backend (which wraps ENS resolver calls).
- If credit score > threshold (e.g., 700), provider agrees; else, requests higher collateral.
- Consumer locks deposit (e.g., 50 USDC) on-chain via wallet signature, opening Nitrolite channel.

- Website UI: Real-time feed showing session initiations, with expandable cards for details. Developers can filter by agent or status, with charts showing credit history trends.

Soft-Settle Streaming (Off-Chain Interaction):

- Provider delivers work units via direct P2P or website-related WebSocket.
- Consumer signs micro-authorization and sends back.
- Loop continues until work completes; all off-chain for speed.
- Website UI: Live monitoring page with progress bars, similar to Stripe's payment dashboards. Alerts for anomalies like signature failures.

Hard Settlement and Closure (On-Chain Finalization):

- Both agents sign final state; submit to Nitrolite contract for fund release.
- If there is a deficit, the consumer's backend triggers Circle Payout from the linked wallet.
- Reputation updated on ENS via contract call.
- Website UI: Settlement confirmation modal (like Uniswap's transaction success popup), updating dashboard with new scores and totals. Exportable reports for audits.

Post-Session Management (On Website):

- Developers review session logs, dispute if needed.
- Agents can query their reputation via API for future interactions.
- UI: Analytics tab with graphs, akin to Dune Analytics' customizable dashboards.

Build Plan-

Tech Stack

- Frontend: React.js + Next.js (for SSR/SEO), Tailwind CSS (responsive), Web3Modal (wallet integration), Ethers.js (contract calls).
- Backend: Node.js/Express, GraphQL (Apollo) for APIs, MongoDB/Postgres for agent registry.
- Blockchain: Hardhat for dev/testing, Solidity 0.8.x for contracts.
- Integrations: Yellow Network SDK, Circle API SDK, ENS.js.
- Tools: Vercel for hosting, Alchemy/Infura for RPC, Sentry for monitoring.

Core Smart Contracts and Backend

- Develop/deploy Nitrolite channel contracts, Reputation Manager.
- Build API endpoints: /register-agent, /init-session, /query-reputation.
- Implement off-chain messaging: WebSocket server for P2P relay.
- Testing: Unit tests (Mocha/Chai), integration tests with Ganache; simulate 100 sessions.

Frontend Development

- Build pages: Landing, Dashboard, Agent Registration, Session Monitor.
- Features: Wallet connect button (top-right, like MetaMask integrations), real-time updates via subscriptions.
- UI Polish: Animations (Framer Motion), charts (Chart.js), themes (dark/light auto-detect).
- Agent API Docs: Swagger UI for programmatic access.

Integrations and Testing

- Hook in Circle/Arc for payouts (API keys in env vars).
- ENS integration for record reads/writes.
- End-to-End Testing: Cypress for UI, load testing with Artillery for scalability.
- Beta: Deploy to testnet.

Value Proposition-

- Enables "pay-per-bit" micro-transactions at internet speeds.
- Reduces trust requirements through collateralized state channels and reputation scores.
- Integrates with existing DeFi tools for seamless adoption.
- Targets agentic commerce, positioning for prizes in pay-per-use apps, agentic commerce, and creative DeFi uses.

Scope-

In Scope:

Identity verification and credit checks using ENS; session initialization with Nitrolite state channels (via Yellow Network); streaming micro-authorizations; final settlements with Circle Payouts and Arc for credit management; reputation updates.

Out of Scope:

Full implementation of AI agent logic (e.g., scraping or research algorithms); arbitration mechanisms for disputes (assumed to be handled by external smart contracts); integration with non-USDC tokens; mobile or web UI for human oversight (focus is on agent-to-agent APIs).

Functional Requirements-

Phase 1: Identity & Credit Check (ENS Integration)

Consumer Agent Authentication

The consumer agent must register/authenticate using an ENS domain.

System: Resolve ENS name to Ethereum address via ENS resolver.

Provider Queries ENS Text Records

Provider agent queries predefined ENS keys.

Logic: If score > 700, reduce collateral requirement.

Decision Logic

If checks pass, proceed to Phase 2; else, reject sessions with error code.

Phase 2: Session Initialization (Yellow Network Integration)

Collateral Lock

Consumer locks deposit in Nitrolite smart contract.

System: Emit event confirming lock; generate session ID.

State Channel Opening

Open bidirectional Nitrolite state channel between consumer and provider addresses.

Initial State: Balance = locked deposit; Counter = 0.

Configuration Parameters

Session parameters: Max duration, micro-payment amount, unit definition.

Phase 3: Soft-Settle Stream (Streaming Micro-Authorizations)

Work Delivery Loop

The provider performs a unit of work.

Sends data hash or payload to consumers via off-chain channel.

Micro-Authorization

Consumer signs off-chain state update: "Shift 0.001 USDC to provider; New balance: X; Nonce: Y".

Signature: Use EIP-712 for typed data signing.

Validation and Continuation

Provider verifies signature against channel state.

If valid, proceed to the next work unit; else, close the session early.

Speed Requirement: Entire loop <100ms.

Monitoring

Both agents track cumulative transfers off-chain.

Phase 4: Hard Settlement (Circle/Arc Integration)

Session Closure

Mutual: Both sign final state (e.g., "Total transferred: 14.50 USDC").

Unilateral: If one party is unresponsive, challenge period (e.g., 10 minutes) before on-chain enforcement.

Fund Release

Nitrolite contract releases funds based on final state.

If deficit (e.g., transfers > deposit), trigger Circle Payout from the consumer's wallet.

Credit Limit Management

Consumer agent queries Circle Wallet balance via Arc API.

Auto-adjust credit limit.

If needed, initiate payout to cover the deficit before closure.

Reputation Update

Post-settlement, update ENS records via Reputation Manager contract.

Score Calculation: e.g., $\text{Score} = (\text{Previous Score} * 0.9) + (100 \text{ if successful, } -200 \text{ if disputed})$.

Technical Architecture-

Core Components

Smart Contracts: Nitrolite Channel (Yellow), Reputation Manager.

Off-Chain: P2P messaging layer (e.g., libp2p).

Integrations: ENS Resolver, Circle API (payouts, wallets), Arc for credit logic.

Data Flow

ENS Query → Credit Check.

On-Chain Lock → Channel Open.

Off-Chain Loop: Work → Sign → Validate.

Final Sign → On-Chain Settle → ENS Update.

Target Audience-

Primary Users:

AI agents in B2B networks, such as researcher agents (consumers) hiring scraper agents (providers).

Secondary Users:

Developers building agentic commerce apps; blockchain integrators using Yellow Network, Circle/Arc, and ENS.

User Personas-

Consumer Agent (e.g., AI Researcher)

- Role: Requests services like data scraping.
- Needs: Prove creditworthiness quickly; stream micro-payments without on-chain delays; manage credit limits autonomously.
- Pain Points: High gas fees for frequent transactions; risk of overpaying or service denial.

Provider Agent (e.g., AI Scraper)

- Role: Delivers granular work (e.g., per URL scraped).

- Needs: Verify consumer's ability to pay; receive incremental authorizations; secure final payout.
- Pain Points: Risk of non-payment after work; slow confirmations disrupting service flow.

Reputation Manager (Smart Contract)

Role: Automated entity updating ENS records.

Needs: Access to settlement data for score calculations.

Assumptions and Dependencies-

Assumptions:

Agents have access to Ethereum-compatible wallets; USDC is the primary settlement token; Network latency is under 100ms for off-chain communications.

Dependencies:

Yellow Network for state channels; Circle/Arc APIs for payouts and wallet management; ENS for domain-based records; Ethereum mainnet or compatible L2 for on-chain actions.